

SKY130 TERNARY STANDARD CELL LIBRARY

Complete Project Plan & Technical Specification

From Device Physics to GDSII Tapeout

6 Phases

Structured milestones

40 Weeks

Estimated timeline

~50 Cells

Target library size

AUTHOR

Roberto Williams Batista

VERSION

0.3

DATE

March 22nd, 2026

EMAIL

robertobatista@ieee.org

WEB

robertowilliams.com

Executive Summary

This document defines the complete project plan for designing, characterizing, and validating an open-source ternary standard cell library targeting the SkyWater SKY130 130nm CMOS process. The library will enable ternary (three-valued) digital circuits to be synthesized and physically implemented using the same OpenLane/OpenROAD toolchain already established for binary designs.

Ternary logic encodes three states — commonly $\{-1, 0, +1\}$ (balanced) or $\{0, 1, 2\}$ (unbalanced) — per signal. The critical insight enabling this on SKY130 is the exploitation of multi-threshold voltage transistor stacking: by combining LVT, SVT, and HVT-equivalent devices (via W/L ratio and body bias tuning) within a single cell, a stable intermediate voltage level can be maintained on an output node, producing the third logic state entirely within standard CMOS technology and without any process modification.

This is the first known attempt to build a fully characterized, OpenLane-compatible ternary standard cell library for SKY130. The work constitutes original research with direct publication potential.

Project Goals

- Design at least 50 ternary standard cells covering inverters, gates, adders, multiplexers, and sequential elements
- Achieve DRC-clean and LVS-clean layouts in SKY130A using Magic
- Produce a fully characterized Liberty (.lib) file across 7 PVT corners using Ictime/OpenSTA
- Produce LEF abstract views compatible with OpenLane 2 / OpenROAD place-and-route
- Validate the library by synthesizing and taping out a small ternary proof-of-concept SoC macro via the Efabless Open MPW or Tiny Tapeout program
- Release the library as open source under Apache 2.0

Project Timeline Overview

Ph.	Phase Title	Timeline	Key Tools	Exit Criteria
1	Environment & Theory	Weeks 1–3	ngspice, Magic, OpenLane	SKY130 device models validated
2	Primitive Cell Design	Weeks 4–9	Magic, ngspice, Xschem	STI / NTI / PTI layouts + SPICE
3	Combinational Cell Library	Weeks 10–18	Magic, Ictime, OpenLane	ternary_std_cell v0.1 LIB/LEF/GDS
4	Sequential Cell Library	Weeks 19–25	Magic, ngspice, OpenSTA	T-DFF, T-Latch GDS + timing
5	Characterization & Sign-off	Weeks 26–31	Ictime, OpenSTA, Calibre	Full .lib (7-corner), DRC/LVS clean
6	Integration & Test Chip	Weeks 32–40	OpenLane 2, Caravel	GDSII submitted for tapeout

PHASE 1 Environment Setup & Theoretical Foundation

🕒 Weeks 1–3

Before drawing a single transistor, the development environment must be reproducible and the ternary device physics must be validated in simulation. This phase has no layout work; it is pure setup and research.

Step 1.1 — Toolchain Installation

Step 1.1 Install and Verify All EDA Tools

Purpose	Build a reproducible, containerized design environment for SKY130 ternary work.
OS Target	Ubuntu 22.04 LTS (bare metal or Docker). All tool versions must be pinned.
Core Tools	Magic 8.3.x, ngspice 39+, Xschem 3.x, KLayout 0.28+, OpenLane 2.x, NetGen 1.5.x, OpenSTA 2.x
PDK Setup	Install open_pdks → sky130A. Confirm: \$PDK_ROOT/sky130A/libs.ref and libs.tech are populated.
Validation	Run a binary inverter simulation in ngspice using sky130_fd_pr__nfet_01v8 and sky130_fd_pr__pfet_01v8 SPICE models. Confirm VTC plot.

Installation Commands

```
# Clone open_pdks and build sky130A
git clone https://github.com/RTimothyEdwards/open_pdks.git
cd open_pdks && ./configure --enable-sky130-pdk
make && make install

# Install Magic
git clone https://github.com/RTimothyEdwards/magic.git
cd magic && ./configure && make && sudo make install

# Install ngspice (must have OSDI/OpenVAF for advanced models)
sudo apt install ngspice
ngspice --version # must be >= 39
```

Step 1.2 — Ternary Theory & Design Strategy

Step 1.2 Define the Ternary Logic Family

Logic Encoding	Use unbalanced ternary {0, VDD/2, VDD} mapped to {0, 1, 2}. Supply: VDD = 1.8V, mid-rail VM = 0.9V.
Vt Strategy	Three stable output levels are achieved by stacking transistors with different effective Vth. SKY130 transistors are tuned via W/L ratios and careful resistive dividers to produce a stable VM at 0.9V ± 50mV.

Key Primitives	Define the three unary ternary inverters: STI (Standard), NTI (Negative), PTI (Positive). All other cells compose from these.
Truth Tables	Define and document complete truth tables for all planned cells: TNAND, TNOR, TXOR, TSUM, TCARRY, TMUX, T-DFF, T-Latch.
Reference	Study: Zhao et al. 2024 (IEEE TETC), Ko et al. 2023 (IEEE TCAS-I), Park et al. 2025 (IEEE Access) for RTL synthesis methodology.

SKY130 does not have native multi-Vt flavors like advanced nodes. The ternary intermediate state is synthesized by exploiting pull-up / pull-down current ratios between PMOS and NMOS networks. This requires careful SPICE modeling before any layout begins.

Step 1.3 — ngspice Model Validation

Step 1.3 Validate the Intermediate Voltage Level in SPICE	
Goal	Confirm that a 3-transistor STI schematic produces a stable VM = 0.9V at its output when input is at VDD/2.
SPICE Models	Use sky130_fd_pr__nfet_01v8 (W=1u, L=0.15u) and sky130_fd_pr__pfet_01v8 (W=2u, L=0.15u) as base. Vary W/L to tune VM.
Simulation	DC sweep VIN from 0 to 1.8V. Overlay VOUT. Identify stable ternary windows. Run parametric sweep on W_NMOS_MID.
PVT Sweep	Simulate at corners: TT/1.8V/27C, SS/1.62V/-40C, FF/1.98V/85C. Confirm VM stays within 0.9V ± 100mV at all corners.
Deliverable	SPICE netlist ternary_STI_validation.spice + DC sweep plots showing all three stable regions.

STI Schematic Concept (SPICE subckt)

```
.subckt SKY130_STI A Y VDD VSS
* PMOS pull-up network (high-Vt equivalent via W/L)
MP1 Y A VDD VDD sky130_fd_pr__pfet_01v8 W=1.0u L=0.15u
MP2 Y A VDD VDD sky130_fd_pr__pfet_01v8 W=0.5u L=0.30u
* Middle state NMOS
MM1 Y A MID VSS sky130_fd_pr__nfet_01v8 W=0.5u L=0.30u
* NMOS pull-down network
MN1 MID A VSS VSS sky130_fd_pr__nfet_01v8 W=1.0u L=0.15u
.ends SKY130_STI
```

Phase 1 Deliverables

Deliverable	Format	Tool / Source
Validated STI SPICE netlist (all PVT corners)	ternary_STI_validation.spice	ngspice

DC sweep plots showing 3 stable logic levels	DC_sweep_PVT.pdf	ngspice + matplotlib
Ternary truth tables document (all 50+ cells)	truth_tables.xlsx	Manual / Excel
Tool version lockfile (pinned versions)	tools_manifest.txt	Bash / env
Docker/environment setup script	setup_env.sh	Bash

PHASE 2

Primitive Cell Design (Inverters & Buffers)

🕒 Weeks 4–9

The three ternary inverters — STI, NTI, and PTI — are the fundamental primitives of the entire library. Every combinational cell is composed from them. Getting these three cells physically correct, DRC-clean, and characterizable is the critical foundation. Do not proceed to Phase 3 until all three inverters pass all verification steps.

Step 2.1

Schematic Capture in Xschem

Step 2.1 Draw All Primitive Schematics	
Cells	STI (Standard Ternary Inverter), NTI (Negative Ternary Inverter), PTI (Positive Ternary Inverter), TBUF (Ternary Buffer = 2x STI in series)
Tool	Xschem with sky130A symbol library
STI Function	Output cycles 2→1→0 as input goes 0→1→2. The three-way inversion.
NTI Function	Output is 0 when input is 0 or 1, output is 1 when input is 2. Fires only on high trit.
PTI Function	Output is 1 when input is 0, output is 0 when input is 1 or 2. Fires only on low trit.
SPICE export	Export .spice netlist from Xschem for each cell. Validate each exported netlist in ngspice before layout.

Step 2.2

Cell Layout in Magic

Step 2.2 Physical Layout of Primitive Cells	
Grid	Use sky130_fd_sc_hd site: 0.46um x 2.72um (9 metal1 tracks). All cell boundaries must snap to this grid.
Cell Height	Fix height at 2.72um for all cells. Power rails VDD (met1, top) and VSS (met1, bottom) must be at exact y=2.72um and y=0.
Well Taps	P-well tap (VPWR port on nwell) and N-well tap (VGND port on pwell) must be defined per the SKY130 tapless cell convention.
Pins	All input/output pins must land on li1 (local interconnect) at grid-aligned positions. Use met1 pins for power.
DRC	Run Magic DRC after every major geometry addition. Target: zero DRC errors before proceeding to LVS.
Layer stack	Use only: ndiff, pdiff, poly, nwell, li1, met1. Avoid met2+ in standard cells. nwell extends full cell height.

Magic Layout Workflow

```
# Launch Magic with sky130A tech
magic -T $PDK_ROOT/sky130A/libs.tech/magic/sky130A.tech &
```

```
# In Magic TCL console – set grid for HD cells
grid 0.46um 0.34um 0.23um 0.17um
snap int

# After layout, extract SPICE for LVS
extract all
ext2spice cthresh 0 rthresh 0
ext2spice
```

Step 2.3 — DRC Verification

Step 2.3 Run and Clear All DRC Violations

Tool	Magic DRC (primary), KLayout DRC with sky130A DRC deck (secondary confirmation)
Common Issues	nwell spacing violations, poly-to-diffusion spacing, metal1 minimum area, li1 via enclosure, nsdm/psdm extension.
PSDM/NSDM	CRITICAL: Extend psdm.drawing and nsdm.drawing layers to the edges of the abutment box using KLayout post-processing to avoid DRC errors at cell boundaries.
Abutment Test	Place two copies of each cell side-by-side in Magic and run DRC on the combined layout. Must be clean at all orientations: 0deg, mirror-X, mirror-Y.
Pass Criteria	Zero DRC errors in both Magic and KLayout DRC for all cell orientations.

KLayout DRC Command

```
klayout -b -r $PDK_ROOT/sky130A/libs.tech/klayout/sky130A.drc \
-rd input=sky130_ternary_STI.gds \
-rd report=STI_drc.xml
```

Step 2.4 — LVS Verification

Step 2.4 Layout vs. Schematic Check

Tool	NetGen (open source) for LVS comparison
Reference	Xschem-exported .spice netlist from Step 2.1
Extracted	Magic ext2spice output from Step 2.2
Process	Run netgen lvs on each cell. Resolve all mismatches: missing connections, swapped nets, extra devices.
Pass Criteria	NetGen reports 'Netlists match uniquely' for all primitive cells.

NetGen LVS Command

```
netgen -batch lvs \
'{sky130_ternary_STI.spice sky130_ternary_STI}' \
'{sky130_ternary_STI_extracted.spice sky130_ternary_STI}' \
```

```
$PDK_ROOT/sky130A/libs.tech/netgen/sky130A_setup.tcl
```

Step 2.5 — Post-Layout Simulation

Step 2.5 Validate Ternary Behavior After Layout

Goal	Confirm that the extracted netlist (with parasitic RC) still produces clean three-level switching.
Test	Apply input sequence 0V → 0.9V → 1.8V → 0.9V → 0V at 100MHz. Measure output VM stability and switching times.
Metrics	Tpd_HL (0→1 transition), Tpd_LH (1→0), VM_stability (max deviation from 0.9V during hold), noise margin.
All PVT	Run at TT/1.8V/27C, SS/1.62V/-40C, FF/1.98V/85C, FS, SF, SF/85C, SS/-40C (7 corners)
Deliverable	post_layout_sim_STI.pdf showing waveforms at all 7 corners, with VM labeled.

Phase 2 Deliverables

Deliverable	Format	Tool / Source
STI, NTI, PTI, TBUF GDS layouts (DRC clean)	sky130_ternary_*_v1.gds	Magic
Xschem schematics (.sch) for all primitives	*.sch	Xschem
Extracted SPICE netlists for LVS	*_extracted.spice	Magic ext2spice
LVS reports (all cells passing)	lvs_report_*.txt	NetGen
Post-layout simulation waveforms (7 corners)	post_layout_sim_*.pdf	ngspice
Abutment DRC test result (all clean)	abutment_drc_report.xml	KLayout

PHASE 3

Combinational Cell Library

 Weeks 10–18

With the three primitives verified, Phase 3 systematically builds the complete combinational cell set. Each cell follows the same workflow: schematic → SPICE simulation → layout → DRC → LVS → post-layout sim. The cells are organized into functional groups.

Step 3.1 — Basic Logic Gates

Step 3.1 TNAND, TNOR, TAND, TOR, TXOR, TXNOR	
TNAND2	2-trit NAND. PMOS pull-up network in parallel, NMOS pull-down in series. Output is STI(min(A,B)) for balanced logic.
TNOR2	2-trit NOR. PMOS in series, NMOS in parallel. Output is STI(max(A,B)).
TAND2	Composition: TBUF(TNAND2(A,B)) — pass through STI to restore correct polarity.
TOR2	Composition: TBUF(TNOR2(A,B)).
TXOR2	TSUM function — critical for adder construction. Requires a 6-transistor custom topology.
Drive strengths	Implement each gate in x1, x2, x4 drive strengths by scaling output-stage PMOS/NMOS widths.

Step 3.2 — Ternary Multiplexers

Step 3.2 TMUX21, TMUX31, TDEMUX13	
TMUX21	2:1 ternary multiplexer — the most used cell in ternary synthesis. Select S=0→Y=A, S=1→Y=B, S=2→Z (high-Z or defined). Required for LUT-style implementation.
TMUX31	3:1 ternary mux — native 3-input selection with 1-trit select. Uniquely efficient in ternary vs. binary.
TDEMUX13	1:3 ternary demultiplexer. Routes a trit to one of three outputs based on a 1-trit select.
Key Note	TMUX21 is the universal ternary logic primitive (equivalent to binary NAND). The synthesis tool will use it heavily. Optimize for minimum area at x1 and maximum speed at x4.
Layout Note	Use transmission gate topology for pass-through paths to minimize VM degradation through the mux stack.

Step 3.3 — Ternary Arithmetic Cells

Step 3.3 THADD (Half Adder), TFADD (Full Adder), TENC, TDEC

THADD	Ternary half adder: inputs A, B (trits). Outputs TSUM (ternary sum) and TCARRY (0 or 1 binary carry). Core of all arithmetic.
TFADD	Ternary full adder: A, B, Cin. Outputs TSUM, COUT. Two stage: first THADD(A,B), then THADD(partial_sum, Cin). Uses ANY gate for carry merge.
TANY	ANY gate: output is 1 if any input is non-zero. Key for carry propagation in ternary adder trees.
TENC / TDEC	Ternary encoder (binary pair → trit) and decoder (trit → binary pair). Required for binary interface boundary cells.
Reference	Follow the adder architecture from Zhao et al. 2024 (IEEE TETC) for Wallace tree multiplier compatibility.

Step 3.4 — Comparison & Threshold Cells

Step 3.4 TCMP, TGTE, TLTC, TCLAMP	
TCMP	Ternary comparator: outputs -1, 0, or +1 based on comparison of two trits. Used in sorting networks and ternary CAM.
TGTE / TLTC	Greater-than-or-equal / less-than comparators for building ternary magnitude comparators.
TCLAMP	Clamps a multi-valued signal to {0, 1, 2}. Used at input ports receiving analog-approximate signals.
TZERO / TONE / TTWO	Constant cells outputting 0, VM, or VDD. Required by synthesis tool for tie cells.

Step 3.5 — Level Shifter & Interface Cells

Step 3.5 Binary-to-Ternary Interface Cells	
B2T	Binary-to-Ternary converter: takes a 2-bit binary pair (B1:B0) and outputs a single trit {0,1,2}. Uses TMUX21 and level-set logic.
T2B	Ternary-to-Binary converter: outputs two binary signals from one trit. Required for interfacing ternary data bus to binary peripherals.
TPADOUT	Ternary output pad driver: drives a 3-level signal to an IO pad. Must handle the 1.8V / 0.9V / 0V swing to the PAD net.
TISOHI / TISOLOW	Isolation cells that clamp a ternary net to binary 0 or 1 during power gating. Required for mixed binary/ternary domain crossings.
Critical Note	These interface cells define the boundary of the ternary compute domain. Every ternary SoC macro will instantiate them at its periphery.

Phase 3 Target Cell Count

Cell Group	Count	Drive Strengths
------------	-------	-----------------

Inverters (STI, NTI, PTI, TBUF)	4	x1, x2, x4
Basic Gates (TNAND, TNOR, TAND, TOR, TXOR)	10	x1, x2, x4
Multiplexers (TMUX21, TMUX31, TDEMUX13)	6	x1, x2, x4
Arithmetic (THADD, TFADD, TANY, TENC, TDEC)	5	x1, x2
Comparison & Threshold	4	x1
Interface & Level Shifters	6	x1
Tie / Filler / Decap Cells	5	—
TOTAL	~50 unique	~130 with drive strengths

Phase 3 Deliverables

Deliverable	Format	Tool / Source
All combinational cell GDS (DRC/LVS clean)	sky130_ternary_*.gds (~40 files)	Magic
SPICE netlists for all combinational cells	sky130_ternary_*.spice	Magic ext2spice
LVS pass reports for all cells	lvs_reports/*.txt (all PASS)	NetGen
Post-layout timing simulation (transient)	transient_sims/*.pdf	ngspice
Merged GDS for entire combinational library	sky130_ternary_comb_v0.1.gds	KLayout merge

PHASE 4 Sequential Cell Library Weeks 19–25

Sequential ternary cells — flip-flops, latches, and scan elements — are significantly more complex than combinational cells due to the need for stable multi-state memory and feedback networks. This phase requires the most careful analog-aware layout.

Step 4.1 — Ternary Latch

Step 4.1 T-Latch: Transparent Level-Sensitive Storage

Function	When enable EN=1 (high trit): Q follows D. When EN=0: Q holds last value. Three stable storage states {0, VM, VDD}.
Topology	Cross-coupled TNOR2 pair for the storage element. TMUX21 for enable muxing. Requires careful sizing to overcome metastability at VM.
VM Stability	The intermediate state VM=0.9V is the hardest to hold. Minimum hold current from cross-coupled inverters must overcome leakage. Size PMOS/NMOS for worst-case SS/-40C corner.
Layout	Feedback paths must be short (use li1, avoid met1 long routes). Place cross-coupled TNOR pair with shared diffusion for compactness.
Simulation	Confirm state retention at VM for 10ns at SS/-40C with no input. Check all 9 D×EN combinations (3 data × 3 enable values).

Step 4.2 — Ternary D Flip-Flop

Step 4.2 T-DFF: Rising-Edge Triggered Three-State Register

Function	Captures D on the rising edge of CLK. Q holds until next rising edge. The workhorse sequential element of ternary datapaths.
Architecture	Master-slave configuration: two T-Latches in series with complementary enable. Master transparent on CLK=0, slave transparent on CLK=1.
Setup/Hold	Define setup time Tsetup and hold time Thold for all input transitions: 0→1, 1→2, 2→0, and reverse. 9 transition pairs total per corner.
Metastability	The VM state is naturally at the metastability boundary of the cross-coupled inverters. Characterize mean time between failures (MTBF) at VM input and report.
Variants	T-DFFR (with async reset to 0), T-DFFS (with async set to 2), T-DFF-SE (scan enable input for test). All three must be built.
Area Budget	Target: < 12um ² per T-DFF cell. Reference: sky130_fd_sc_hd_dfxt_1 is ~4.5um ² for binary. Ternary will be 2-3x larger due to extra state.

Step 4.3 — Scan Chain Infrastructure

Step 4.3 T-DFF-SE and Ternary Scan Chain

T-DFF-SE	Scan-enable DFF: when SE=1, data input shifts from SI (scan in) instead of D. Required for manufacturing test and debug.
Scan Encoding	Ternary scan data is 2 bits per trit in the scan chain (binary scan domain). T-DFF-SE must encode/decode at its scan boundary.
Boundary Scan	Define a ternary boundary scan register format compatible with binary JTAG infrastructure. Document the trit-to-bit encoding used.
Deliverable	Testbench demonstrating a 4-cell ternary scan chain shifting through all 81 trit combinations (3^4) correctly.

Step 4.4 — Ternary Clock Cells**Step 4.4 TCLKBUF, TCLKGAT (Clock Gating Cell)**

TCLKBUF	High-drive clock buffer. Same function as TBUF but characterized with clock input slew constraints. Must have matched rise/fall delay.
TCLKGAT	Integrated clock gating cell: clock enable + latch + clock buffer in one cell. Reduces dynamic power by preventing unnecessary trit transitions.
Clock Domain	The clock in a ternary SoC remains binary (0/1). Only data is ternary. TCLKBUF feeds binary clock to T-DFF CLK inputs — the library must be clear about this.
Constraint	Max output capacitance: 100fF. Max input slew: 500ps. These become constraints in the Liberty file.

Phase 4 Deliverables

Deliverable	Format	Tool / Source
T-Latch GDS (DRC/LVS clean)	sky130_ternary_tlatch.gds	Magic
T-DFF, T-DFFR, T-DFFS, T-DFF-SE GDS	sky130_ternary_tdff_*.gds	Magic
TCLKBUF, TCLKGAT GDS	sky130_ternary_tclk*.gds	Magic
Setup/hold characterization (9 transitions × 7 corners)	seq_timing_report.xlsx	ngspice
Scan chain testbench and waveforms	scan_chain_tb/*.pdf	ngspice
Metastability analysis report	metastability_analysis.pdf	ngspice + analysis

PHASE 5 Characterization, Liberty Generation & Sign-Off Weeks

26–31

Characterization is the process of extracting timing and power data from SPICE simulations and encoding it into the Liberty (.lib) format that synthesis tools consume. This is the most computationally intensive phase and requires scripting to handle the combinatorial explosion of input conditions across all cells, corners, and slew/load combinations.

Step 5.1 — Define Characterization Matrix

Step 5.1 PVT Corners and Slew/Load Tables

7 PVT Corners	TT/1.8V/27C (typical), SS/1.62V/-40C (worst power/speed), FF/1.98V/85C (best speed), SF/1.8V/27C, FS/1.8V/27C, SS/1.62V/85C, FF/1.98V/-40C
Input Slews	7 values: 0.01, 0.05, 0.1, 0.2, 0.5, 1.0, 2.0 ns
Output Loads	7 values: 0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0 pF
Per-cell Simulations	For a 2-input gate: 3 arcs × 7 slews × 7 loads × 7 corners = ~1,029 SPICE runs per cell. With ~130 cells = ~133,000 total simulations.
Compute	Parallelize using GNU Parallel or a Python multiprocessing pool. Estimate 30-60 min per cell on a modern workstation.

The ternary library requires extending the Liberty format to handle 3-valued logic arcs. Standard Liberty only defines timing arcs for binary transitions. A custom Python post-processor will augment the .lib with extra arcs for 0→2 and 2→0 ternary transitions.

Step 5.2 — Liberty File Generation

Step 5.2 Generate .lib Files Using Ictime or Custom Scripts

Tool	Ictime (LibreCores characterization tool) — open source, supports custom cells. Fork and modify to handle 3-level ternary arcs.
Inputs	Extracted SPICE netlist (.spice), PDK model cards (.lib SPICE), cell LEF (for pin capacitance extraction)
Outputs	sky130_ternary_tt_025C_1v80.lib — one .lib per corner. 7 corners total.
Timing Arcs	For each input→output arc: cell_rise, cell_fall, rise_transition, fall_transition. For ternary: also define VM_rise (0→VM) and VM_fall (VM→0) arcs.
Power	internal_power, leakage_power. Measure average current during switching for dynamic power. Measure quiescent current for leakage at each output state {0, VM, VDD}.
Validation	Run OpenSTA on a test netlist using the generated .lib. Confirm timing reports match raw ngspice waveform measurements within 10%.

Ictime invocation (per cell per corner)

```
lctime --liberty-in sky130_ternary_template.lib \  
--cell sky130_ternary_STI \  
--spice sky130_ternary_STI.spice \  
--pdk-models $PDK_ROOT/sky130A/models/corners/tt.spice \  
--output-liberty sky130_ternary_tt_025C_1v80.lib
```

Step 5.3 — LEF Abstract View Generation

Step 5.3 Generate LEF Files for Place-and-Route	
Tool	Magic abstract command — generates LEF from layout. One LEF per cell, merged into a single library LEF.
LEF Content	MACRO block with: CLASS CORE, SITE unithd, SIZE (width × 2.72um), PIN definitions with DIRECTION and PORT geometry, OBSTRUCTION layers.
Pin Layers	All signal pins on li1 at grid intersections. Power pins (VPWR/VGND) on met1. No met2+ obstructions in standard cells.
Antenna Rules	Add ANTENNAGATEDIODE and ANTENNADIFFAREA to each input/output pin per SKY130 antenna rules. Required to pass antenna DRC.
Validation	Import merged LEF into OpenROAD. Confirm all cells load cleanly: no pin overlaps, correct SITE assignment, no orphaned ports.

Magic LEF Export

```
# In Magic TCL console  
load sky130_ternary_STI  
lef write sky130_ternary_STI.lef  
  
# Merge all cell LEFs into one library LEF  
cat sky130_ternary_*.lef > sky130_ternary_merged.lef
```

Step 5.4 — Final DRC Sign-Off

Step 5.4 Complete DRC on Merged Library GDS	
Tool	KLayout DRC with the full sky130A DRC rule deck (all rules enabled, including antenna rules)
Scope	Run DRC on every cell individually AND on the merged GDS with all cells tiled in a test array.
Antenna DRC	Run antenna DRC rule check. Fix any antenna violations by adding ANTENNADIODE cells or modifying routing layers in the cell.
Manufacturability	Verify all cells pass the complete 500+ DRC rule set. Document any known waivers with justification (rare in std cells).
Pass Criteria	Zero DRC errors, zero LVS errors for ALL cells in the library. This is a hard requirement for tapeout.

Phase 5 Deliverables

Deliverable	Format	Tool / Source
Liberty files (7 PVT corners)	sky130_ternary_*.lib (7 files)	Icetime
Merged library LEF	sky130_ternary.lef	Magic abstract
Merged library GDS	sky130_ternary_lib_v1.0.gds	KLayout
Full characterization database	char_db/*.csv (all cells × corners)	ngspice + scripts
OpenSTA timing validation report	sta_validation_report.txt	OpenSTA
Final DRC sign-off report (zero violations)	final_drc_signoff.xml	KLayout
Final LVS sign-off report (all PASS)	final_lvs_signoff.txt	NetGen

PHASE 6 Integration — Ternary Proof-of-Concept SoC Macro
Weeks 32–40

A standard cell library without a real design that uses it is unvalidated. Phase 6 implements a small but complete ternary functional block — a ternary adder tree or ternary neural network multiply-accumulate unit — and synthesizes, places, and routes it using OpenLane 2 with the newly characterized library. This macro is submitted for fabrication via Efabless Open MPW or Tiny Tapeout.

Step 6.1 — Ternary RTL Design

Step 6.1 Write RTL for a Ternary MAC Unit

Design Choice	Ternary 4-trit × 4-trit multiplier-accumulate (MAC) unit. Inputs: A[3:0], B[3:0] as trits (2-bit encoded). Output: P[7:0] accumulated product (trit encoded).
Why This Design	A ternary MAC directly demonstrates the density advantage of ternary: 4-trit inputs carry $\log_2(3^4)=6.34$ bits vs. 4 binary bits. Relevant to AI inference acceleration and your VectaDB work.
RTL Encoding	Use binary pair encoding internally: each trit is 2 bits {00=0, 01=1, 10=2}. RTL written in Verilog with trit-encoded operations.
Synthesis Flow	Use Yosys with a custom synthesis script mapping ternary RTL to the sky130_ternary cell library. Requires a custom techmap file for ternary primitives.
Simulation	Write a complete testbench covering all $3^8 = 6,561$ input combinations. Run RTL simulation in iverilog/Verilator before synthesis.

Step 6.2 — Custom Synthesis with Yosys

Step 6.2 Map RTL to Ternary Standard Cells

Challenge	Yosys does not natively understand ternary logic. A custom techmap.v file must teach Yosys how to map ternary operations to cells in sky130_ternary.
Approach	Define ternary operators in Verilog using 2-bit trit encoding. Create ABC technology mapping constraints that prefer ternary cells over binary decomposition.
Techmap	Write techmap rules for: trit addition ($\rightarrow$ TFADD), trit comparison ($\rightarrow$ TCMP), ternary mux ($\rightarrow$ TMUX21), ternary register ($\rightarrow$ T-DFF).
Script	Yosys synthesis script: read_verilog $\rightarrow$ synth $\rightarrow$ techmap sky130_ternary_techmap.v $\rightarrow$ dfflibmap sky130_ternary_tt.lib $\rightarrow$ abc $\rightarrow$ write_verilog
QoR Metrics	Report cell count, estimated area (from LEF), and critical path depth. Compare to an equivalent binary implementation.

Yosys Synthesis Script

```
# ternary_synth.ys
```

```
read_verilog ternary_mac.v
hierarchy -check -top ternary_mac
synth -flatten
techmap -map sky130_ternary_techmap.v
dfflibmap -liberty sky130_ternary_tt_025C_1v80.lib
abc -liberty sky130_ternary_tt_025C_1v80.lib -script +strash;scorr;ifraig;retime
clean
write_verilog -noattr ternary_mac_synth.v
```

Step 6.3 — Place and Route with OpenLane 2

Step 6.3 OpenLane 2 Flow Configuration for Ternary Library

Config Keys	STD_CELL_LIBRARY: sky130_ternary EXTRA_LEFS: sky130_ternary.lef EXTRA_LIBS: sky130_ternary_*.lib (all 7 corners)
Floorplan	Aspect ratio 1:1. Utilization ~40% (leave margin for ternary cell size variance). Core area target ~0.05mm².
Power Delivery	Add met1 power straps for VGND (0V) and VPWR (1.8V). No mid-rail (VM=0.9V) power distribution — VM is generated locally within each cell.
Routing	OpenROAD detailed router. Ternary signals use same routing layers as binary. No special rules needed — ternary is transparent to the router.
Timing Closure	Run OpenSTA in the loop. Fix violations by upsizing cells (x1→x2→x4 drive strengths). Target: WNS > -100ps, TNS=0.
DRC/LVS	Run KLayout DRC and NetGen LVS on the final PnR GDSII. Must be clean before tapeout submission.

Step 6.4 — Caravel Integration & Tapeout Submission

Step 6.4 Wrap the Macro in Caravel User Project Wrapper

Platform	Efabless Caravel harness (Open MPW shuttle) or Tiny Tapeout (most accessible for proof of concept).
Wrapper	Instantiate ternary_mac inside user_project_wrapper. Connect trit-encoded inputs/outputs to Caravel wishbone or GPIO buses via T2B/B2T interface cells.
Precheck	Run efabless precheck tool on the final GDSII. Must pass all checks: DRC, LVS, antenna, pin alignment, power connection.
Submission	Submit via the Efabless platform with: GDS, LEF, final timing report, simulation results, and documentation.
Timeline Note	Open MPW shuttles run approximately every 4-6 months. Target the shuttle immediately following Phase 5 completion. Tiny Tapeout runs more frequently and is lower risk for first tapeout.

Phase 6 Deliverables

Deliverable	Format	Tool / Source
Ternary MAC RTL + testbench	ternary_mac.v + tb_ternary_mac.v	Verilog
Yosys techmap for ternary cells	sky130_ternary_techmap.v	Yosys
Synthesized gate-level netlist	ternary_mac_synth.v	Yosys
OpenLane 2 config for ternary library	config.json	OpenLane 2
Final PnR GDSII	ternary_mac_final.gds	OpenROAD
Post-route timing report (WNS, TNS)	timing_signoff.txt	OpenSTA
Tapeout-ready Caravel wrapper GDSII	user_project_wrapper.gds	OpenLane 2 / Caravel
Efabless precheck pass log	precheck_results.log	Efabless precheck

Complete Tool & Resource Reference

Open Source EDA Tools

Tool	Purpose	Repository / Install
Magic 8.3+	Layout editor + DRC + LVS extraction	github.com/RTimothyEdwards/magic
ngspice 39+	SPICE circuit simulation (pre/post layout)	ngspice.sourceforge.io
Xschem 3.x	Schematic capture with sky130 symbols	github.com/StefanSchippers/xschem
KLayout 0.28+	GDSII viewer, DRC runner, LEF/DEF viewer	klayout.de
NetGen 1.5+	LVS (Layout vs Schematic)	github.com/RTimothyEdwards/netgen
OpenLane 2	RTL-to-GDSII flow (synthesis through PnR)	github.com/efabless/openlane2
OpenROAD	Place and route, timing (embedded in OpenLane)	github.com/The-OpenROAD-Project
OpenSTA 2+	Static timing analysis	github.com/The-OpenROAD-Project/OpenSTA
Yosys 0.38+	Logic synthesis, techmap	github.com/YosysHQ/yosys
lctime	Liberty file characterization (open source)	github.com/LibreCores/lctime
open_pdk	PDK installation tool for sky130A	github.com/RTimothyEdwards/open_pdk

Key Reference Papers

- Zhao et al. (2024) — Efficient Ternary Logic Circuits Optimized by Ternary Arithmetic Algorithms. IEEE Transactions on Emerging Topics in Computing, Vol. 12, No. 3. (Multi-Vt CMOS ternary arithmetic, includes STI layout in CMOS180)
- Ko et al. (2023) — Exploration of Ternary Logic Using T-CMOS for Circuit-Level Design. IEEE TCAS-I. (Complete balanced ternary full adder + sequential cells in T-CMOS)
- Park et al. (2025) — An RTL-based General Synthesis Methodology for Device-Independent Ternary Logic Circuits. IEEE Access. (Critical for Step 6.2 Yosys integration)
- Lee et al. (2025) — Ternary Toward Binary: Circuit-level Implementation of Ternary Logic Using Depletion-mode and Conventional MOSFETs. IEEE Access.
- w32agobot/SKY130_standardcelldesign (GitHub) — Custom standard cell tutorial for SKY130. Essential reference for Magic cell layout conventions.
- AvalonSemiconductors/sky130_as_sc_hs (GitHub) — Custom high-speed standard cell library for sky130A. Excellent reference for lctime integration and OpenLane config.

Repository Structure

```
sky130_ternary_stdcell/  
├── docs/           # Design documentation, truth tables
```

```
|— xschem/          # Schematic sources (.sch)
|— spice/           # SPICE netlists (hand-written + extracted)
|— magic/           # Magic layout files (.mag)
|— gds/             # Generated GDS per cell + merged library
|— lef/             # Generated LEF per cell + merged library LEF
|— lib/             # Liberty files (7 PVT corners)
|— char/            # Characterization scripts + raw data
|— drc_reports/     # DRC sign-off reports
|— lvs_reports/     # LVS sign-off reports
|— sim/             # ngspice simulation scripts + waveforms
|— pnr_demo/        # OpenLane 2 demo design (ternary MAC)
|   |— rtl/         # Verilog RTL
|   |— synth/       # Yosys synthesis scripts + techmap
|   |— openlane/    # OpenLane config + run outputs
|— README.md        # Getting started guide
```

Risk Register

Risk	Severity	Mitigation	Contingency
VM = 0.9V unstable at SS/-40C corner	HIGH	Validate in SPICE before layout. Upsize cross-coupled inverter in latch if needed.	Accept $\pm 150\text{mV}$ VM tolerance; restrict to 5-corner lib.
SKY130 lacks native multi-Vt devices	MEDIUM	Use W/L tuning + body bias. Validate against PDK models carefully.	Fall back to 3.3V supply with wider voltage margin for VM.
lctime cannot characterize ternary arcs	MEDIUM	Fork lctime; add ternary arc support. Or write custom Python characterizer using ngspice.	Use manual characterization for first 10 cells to validate flow.
Yosys techmap for ternary is unsupported	MEDIUM	Use binary encoding in Verilog; write explicit techmap for 2-bit trit operations.	Hand-instantiate ternary cells for demo design instead of synthesis.
DRC failures at cell abutment boundaries	MEDIUM	Post-process GDS with KLayout to extend PSDM/NSDM layers. Test abutment DRC early.	Widen cell boundary by 0.1 μm ; update LEF accordingly.
Tapeout shuttle slot unavailable	LOW	Monitor Efabless shuttle schedule. Register early. Have Tiny Tapeout as backup.	Delay tapeout by one shuttle cycle (~6 months). Not a blocker for library release.

Open Source Release Strategy

The library will be released under Apache 2.0, mirroring the license of the SKY130 PDK itself. The GitHub repository will be structured to allow direct use with OpenLane 2 by adding it as an EXTRA_LIBS entry. A companion paper documenting the design methodology, SPICE validation, and characterization results will be submitted to an open-access IEEE journal (recommended: IEEE Access) simultaneously with the first silicon results.

Given the absence of any existing ternary standard cell library for SKY130 or any open PDK, this project occupies genuinely novel research space. Even a partial library (inverters + basic gates + one DFF) fully characterized and tapeout-validated would constitute a significant contribution to the open silicon community.

Connection to VectaDB & AI Inference

The long-term application target — connecting ternary logic to the AI accelerator domain — is strong. Ternary neural networks (TNNs) use weights in $\{-1, 0, +1\}$, which map naturally to balanced ternary trits. A ternary dot-product array built from TFADD and TMUX cells would require roughly 40% fewer cells than an equivalent binary implementation for the same weight precision. This positions the library directly as infrastructure for a future ternary AI accelerator SoC, extending the security camera work from the first SKY130 project into a fundamentally more efficient compute substrate.